07th June 2026



Università degli Studi di Padova

Università Degli Studi di Padova

Cyber-Physical Systems and IoT Security 2025/2026

Final project report

Reference paper: Authentication of IoT Device and IoT Server Using Secure Vaults

Student: Valentina Beni
Student ID: 2155350

Contents

1 Objectives **2**

2 Setup **2**

3 Experiments **2**

3.1 run_secure_vault.py 4

3.2 run_ECC.py 4

4 Results **5**

1 Objectives

The reference paper addresses the limitations of single password-based authentication systems and introduces a new mutual authentication mechanism that relies on multiple keys.
In this approach, a collection of shared keys, referred to as a "secure vault", serves as the shared secret between the IoT device and the server. The contents of this vault are updated after every successful communication session.
The goal of this project is to replicate the authentication mechanism described in the paper and perform measurements of the time needed to complete the authentication process to compare it with the results of the paper. To achieve this, it will be necessary to simulate both an IoT device and a server to facilitate the authentication process.
Additionally the multi-key mechanism will be compared against an ECC-based authentication mechanism.

2 Setup

The original paper implemented the mechanism with an Arduino device, in this project it will be recreated with a local Python code.
In order to execute the code it's recomended to create a virtual environment and installing the libraries listed in the file 'REQUIREMENTS.TXT', following the commands:

```
python -m venv my_venv
.\my_venv\Scripts\activate
pip install -r requirements.txt
```

Once installed the dependencies, it's sufficient to execute the two main scripts to run the simulation and then compare the results.

```
py run_ECC.py
py run_secure_vault.py
```

3 Experiments

The project includes files required for simulating a server, two different IoT devices, a secure vault, and a configuration file used to define the parameters necessary for the simulation.
The following files are used to define the main classes implemented in the simulation:

- **Vault.py**: Class representing secure cryptographic key storage and management system;
- **Server.py**: Class representing the server responsible for device authentication and vault synchronization, using both the secure vault and the ECC mechanism;
- **DeviceVault.py**: Class representing a device that interacts with the server for authentication and vault synchronization;
- **DeviceECC.py**: Class representing a device that uses ECC for authentication with the server.

The project includes two executable files, one for each authentication type:

- **run_ECC.py**
- **run_secure_vault.py**

Both simulations generate an IoT device and a server, then attempt to complete their respective authentication process.
The operations are repeated multiple times according to the parameters defined in the configuration file.
At the end of execution the average results of each process are displayed for comparison.

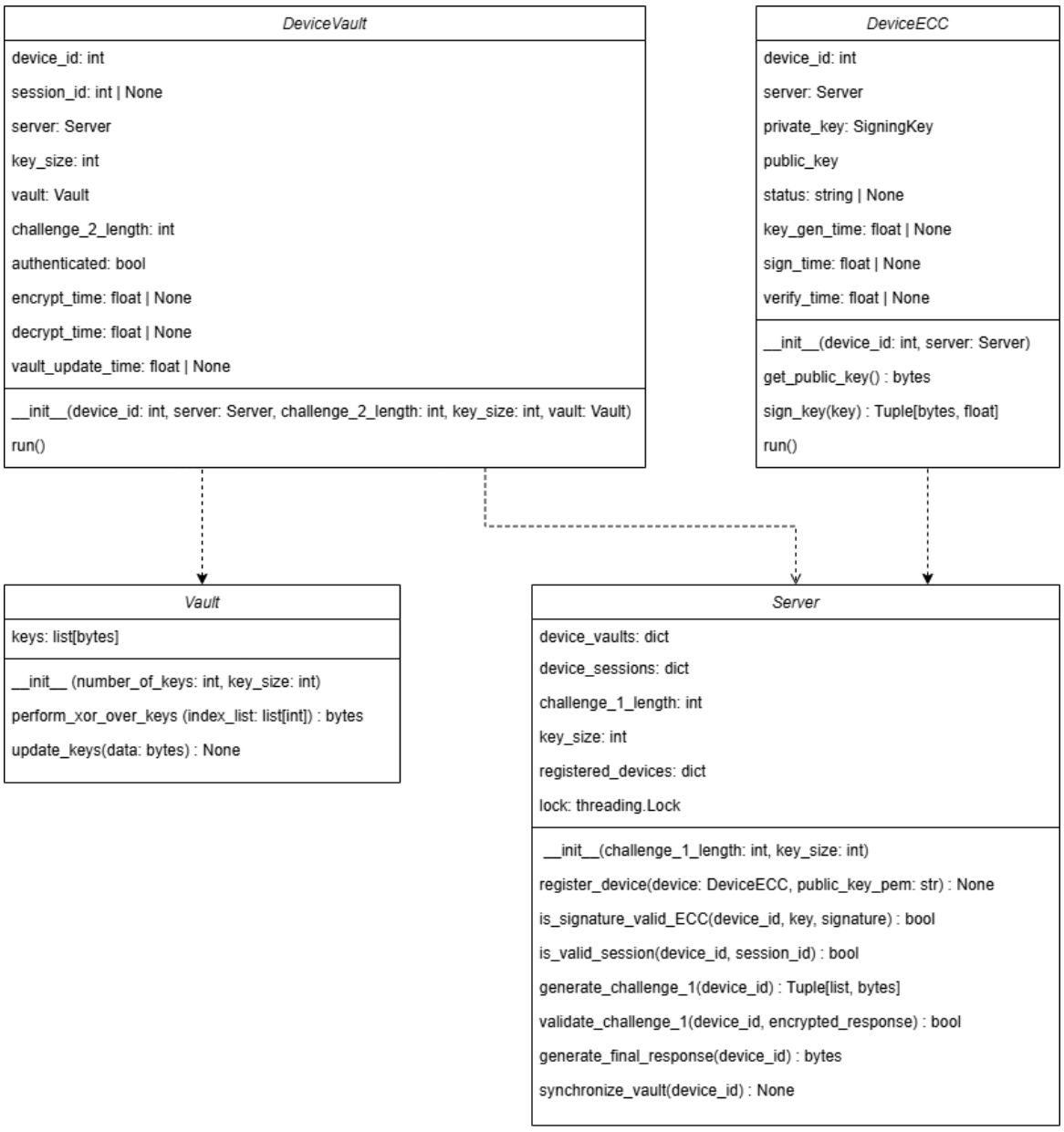


Figure 1: Class diagram

3.1 run_secure_vault.py

The Vault class represents and manages the shared keys used by the Server and DeviceVault objects for authentication. To create a Vault object, the number of keys and their size must be specified by modifying the `.env` file in the project folder. The class functions handle the bitwise xor operation by taking the necessary key indexes as input and update the contents of the secure vault.
The DeviceVault class represents an IoT device that uses the authentication method described in the paper.
As outlined in the paper, the device contacts the server to receive the first challenge C1, solves it, and then sends the results back along with a new challenge C2 for the server. Upon receiving the final response, the device verifies whether it meets the required conditions. If so, it instructs the server to update the vault, completing the authentication process.
The script generates a number of devices, as defined in the configuration file, and performs the authentication process for each one. During this process it records the time required for each step of the mechanism and, at the end, it evaluates the average time taken by the entire process. The results are presented in Table 1.

Table 1: Average performance metrics.

Phase	**Average time (ms)**
Challenge C1	0.24
Challenge C2	0.24
Vault update time	0.13
Total time	**0.61**

3.2 run_ECC.py

The script simulates an IoT authentication system using ECDSA (Elliptic Curve Digital Signature Algorithm), a cryptographic method based on ECC (Elliptic Curve Cryptography). ECC is a public-key cryptography approach that provides robust security with smaller key sizes, making it particularly suitable for resource-constrained IoT devices.
The script creates multiple IoT devices and a central server that registers their public keys. The server generates cryptographic key for each device, which the devices then sign and return for verification.
Like before the script measures the execution times for key generation, signing, and verification to analyze the performance of the authentication process. The results are displayed in Table 2

Table 2: Average performance metrics for ECDSA.

Phase	**Average time (ms)**
Key generation	0.048
Signing	0.77
Verification	2.26
Total time	**3.04**

4 Results

Since energy consumption could not be measured directly, the script results were compared with the data in the paper's "Table 1 Energy consumption of different protocols."
The Secure Vault mechanism script returned different execution times, performing faster than the paper's implementation. This difference can be attributed to the hardware gap since the paper's experiments used an Arduino, whereas the local scripts ran on a high-spec PC.
The same applies to the ECC-based authentication script, which also reported significantly shorter execution times than those in the paper.
Despite these performance differences, the proposed mechanism still demonstrates faster execution than ECC-based authentication.